Artificial Intelligence Practical

UNIVERSITY OF DELHI

Ramanujan College

DSC 16: Artificial Intelligence

Semester-6

(2025-26)

Submitted By:

~Ratnesh Kumar

20231432(23020570034)

Submitted To:

~ Ms. Bhavya Ahuja Grover

Acknowledgement

I would like to take this opportunity to acknowledge everyone who has helped us in every stage of this project.

I am deeply indebted to my Artificial Intelligence Professor, Ms. Bhavya Ahuja Grover for his guidance and suggestions in completing this practical. The completion of this practical was possible under his guidance and support.

I am also very thankful to my parents and my friends who have boosted me up morally with their continuous support.

At last but not least, I am very thankful to God almighty for showering his blessings upon me.

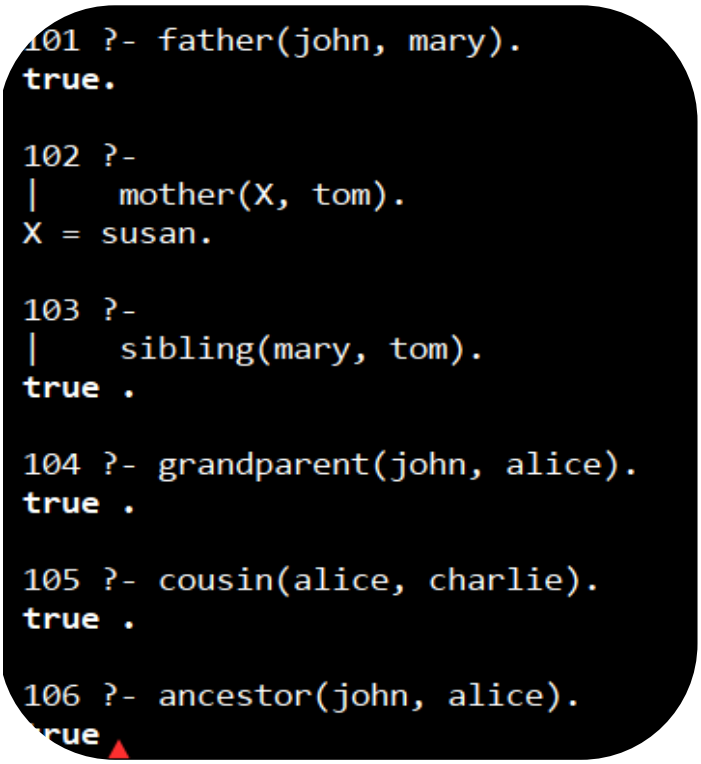
Index

1.Write a PROLOG program to implement the family tree and demonstrate the family relationship.

Code:

```
parent(john, mary).

parent(john, tom).

parent(john, ann).

parent(susan, mary).

parent(susan, tom).

parent(susan, ann).

parent(mary, alice).

parent(mary, bob).

parent(tom, charlie).

parent(tom, diana).

parent(ann, edward).

male(john).

male(tom).

male(bob).

male(charlie).

male(edward).

female(susan).

female(mary).

female(ann).

female(alice).

female(diana).

father(X, Y) :- parent(X, Y), male(X).

mother(X, Y) :- parent(X, Y), female(X).

child(X, Y) :- parent(Y, X).

son(X, Y) :- child(X, Y), male(X).

daughter(X, Y) :- child(X, Y), female(X).

sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.

brother(X, Y) :- sibling(X, Y), male(X).

sister(X, Y) :- sibling(X, Y), female(X).

grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

grandfather(X, Y) :- grandparent(X, Y), male(X).

grandmother(X, Y) :- grandparent(X, Y), female(X).

grandchild(X, Y) :- grandparent(Y, X).

grandson(X, Y) :- grandchild(X, Y), male(X).

granddaughter(X, Y) :- grandchild(X, Y), female(X).

uncle(X, Y) :- brother(X, Z), parent(Z, Y).

aunt(X, Y) :- sister(X, Z), parent(Z, Y).

cousin(X, Y) :- parent(Z, X), parent(W, Y), sibling(Z, W).

ancestor(X, Y) :- parent(X, Y).

ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).

descendant(X, Y) :- ancestor(Y, X).
```

Output:

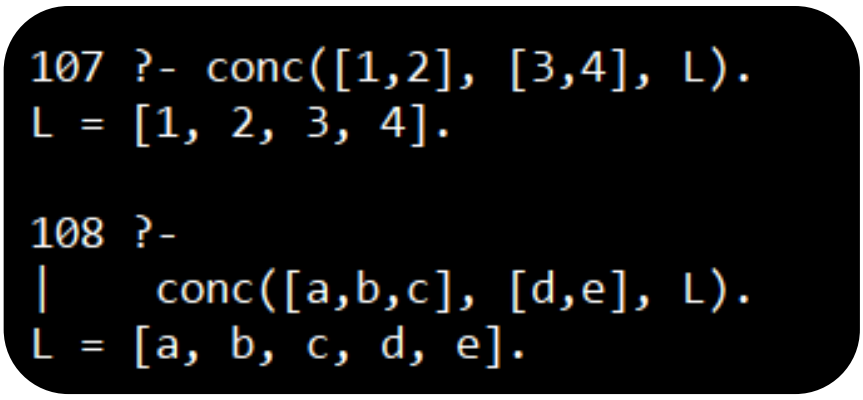


2. Write a PROLOG program to implement conc(L1, L2, L3) where L2 is the list to be appended with L1 to get the resulted list L3.

Code:

```
conc([], L2, L2).

conc([H|T1], L2, [H|T3]) :- conc(T1, L2, T3).
```

Output:

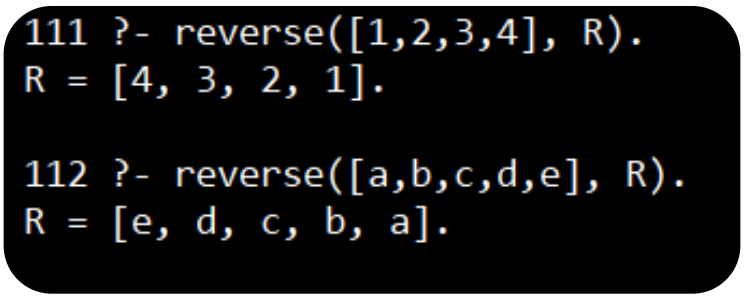


3. Write a PROLOG program to implement reverse(L, R) where List L is original and List R is reversed list.

Code:

```
reverse_list(L, R):-
    rev(L,[],R).
rev([], Acc, Acc).
rev([H|T],Acc, R) :-
    rev(T, [H|Acc], R).
```

Output:

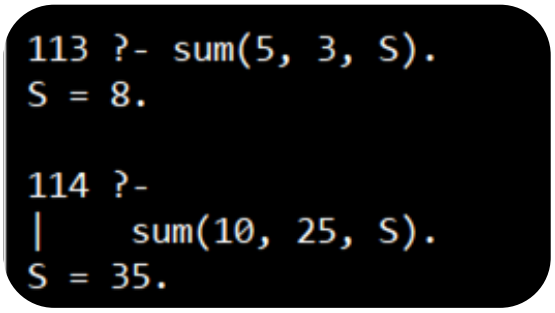


4. Write a PROLOG program to calculate the sum of two numbers.

Code:

```
sum(X, Y, Sum) :- Sum is X + Y.
```

Output:

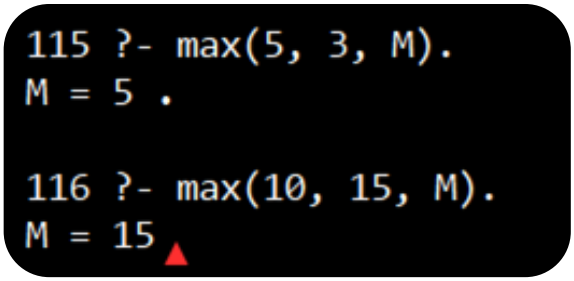


5. Write a PROLOG program to implement max(X, Y, M) so that M is the maximum of two numbers X and Y.

Code:

```
max(X, Y, X) :- X >= Y.

max(X, Y, Y) :- Y > X.
```

Output:

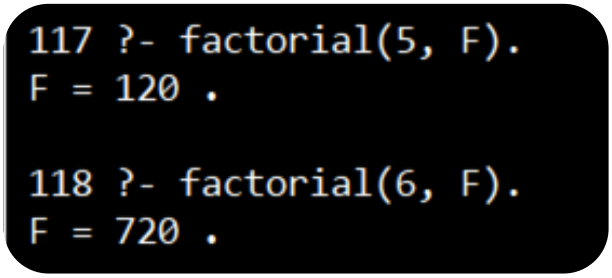


6. Write a program in PROLOG to implement factorial (N, F) where F represents the factorial of a number N.

Code:

```
factorial(0, 1).

factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.
```

Output:

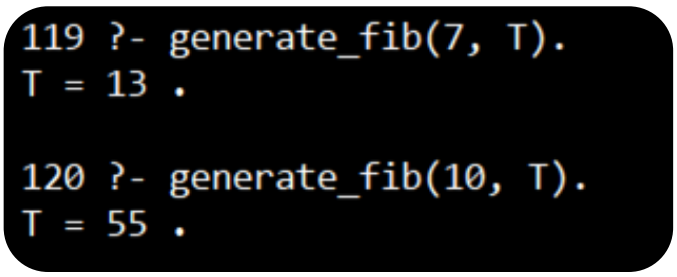


7. Write a program in PROLOG to implement generate_fib(N,T) where T represents the Nth term of the Fibonacci series.

Code:

```
generate_fib(0, 0).

generate_fib(1, 1).

generate_fib(N, T) :-

    N > 1,

    N1 is N - 1,

    N2 is N - 2,

    generate_fib(N1, T1),

    generate_fib(N2, T2),

    T is T1 + T2.
```

Output:

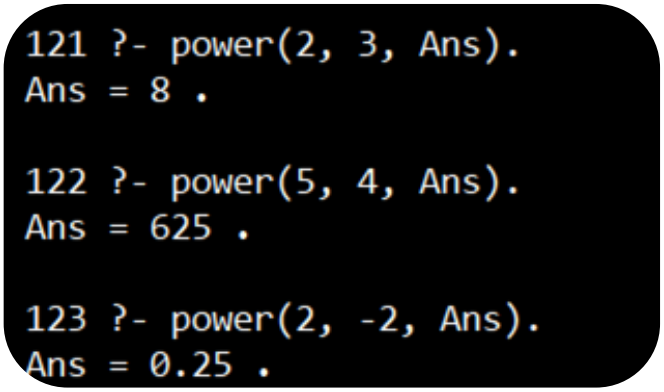


8. Write a PROLOG program to implement power (Num, Pow, Ans) : where Num is raised to the power Pow to get Ans.

Code:

```
power(_, 0, 1).
power(Num, Pow, Ans) :-
    Pow > 0,
    Pow1 is Pow - 1,
    power(Num, Pow1, Ans1),
    Ans is Num * Ans1.
power(Num, Pow, Ans) :-
    Pow < 0,
    PosPow is -Pow,
    power(Num, PosPow, TempAns),
    Ans is 1 / TempAns.
```

Output:

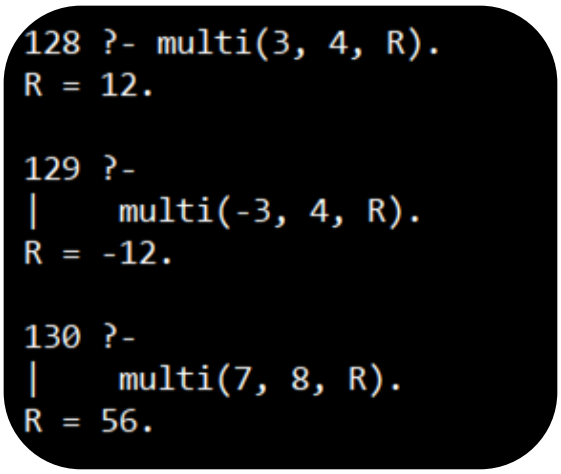


9. PROLOG program to implement multi (N1, N2, R) : where N1 and N2 denotes the numbers to be multiplied and R represents the result.

Code:

```
multi (N1, N2, R) :- R is N1 * N2.
```

Output:

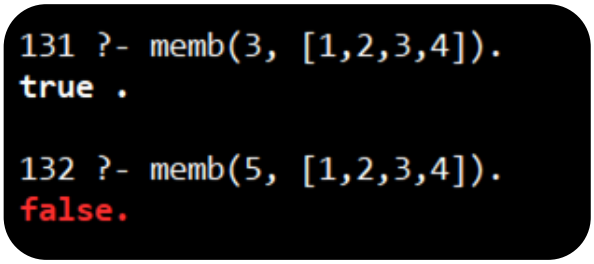


10. Write a PROLOG program to implement memb(X, L): to check whether X is a member of L or not.

Code:

```
memb(X, [X|_]).

memb(X, [_|T]) :- memb(X, T).
```

Output:



11. Write a PROLOG program to implement sumlist(L, S) so that S is the sum of a given list L.

Code:

```
sumlist([], 0).

sumlist([H|T], S) :-

    sumlist(T, S1),

    S is H + S1.
```

Output:

```
133 ?- sumlist([1,2,3,4,5], S).
S = 15.

134 ?-
|    sumlist([10,20,30], S).
S = 60.
```

12. Write a PROLOG program to implement two predicates evenlength(List) and oddlength(List) so that they are true if their argument is a list of even or odd length respectively.

Code:

```
evenlength([]).

evenlength([_|T]) :- oddlength(T).

oddlength([_]).

oddlength([_|T]) :- evenlength(T).
```

Output:

```
135 ?- evenlength([1,2,3,4]).
true .

136 ?- oddlength([1,2,3]).
true .

137 ?- evenlength([a,b]).
true .
```

13. Write a PROLOG program to implement maxlist(L, M) so that M is the maximum number in the list.

Code:

```
max(X, Y, X) :- X >= Y.

max(X, Y, Y) :- Y > X.

maxlist([X], X).

maxlist([H|T], M) :-
    maxlist(T, M1),
    max(H, M1, M).
```

Output:

```
138 ?- maxlist([3,7,2,9,1], M).
M = 9 .

139 ?- maxlist([5,2,8,1,9,3], M).
M = 9 .
```

14. Write a PROLOG program to implement insert(I, N, L, R) that inserts an item I into Nth position of list L to generate a list R.

Code:

```
insert(I, 1, L, [I|L]).

insert(I, N, [H|T], [H|R]) :-
    N > 1,
    N1 is N - 1,
    insert(I, N1, T, R).
```

Output:

```
140 ?- insert(x, 3, [a,b,c,d], R).
R = [a, b, x, c, d] .

141 ?- insert(5, 1, [1,2,3], R).
R = [5, 1, 2, 3] .
```

15. Write a PROLOG program to implement delete(N, L, R) that removes the element on Nth position from a list L to generate a list R.

Code:

```
delete(1, [_|T], T).
delete(N, [H|T], [H|R]) :-
    N > 1,
    N1 is N - 1,
    delete(N1, T, R).
```

Output:

```
142 ?- delete(2, [a,b,c,d], R).
R = [a, c, d] .

143 ?- delete(1, [1,2,3,4], R).
R = [2, 3, 4] .
```